Project Title: *Parking Lot Management System*

Course: Programming in Java

Submitted By: Akshat Saxena

Academic Year: 2025

Institution: VIT Bhopal University

1. Introduction

Modern cities face increasing challenges in managing vehicle parking efficiently. As the number of personal vehicles grows each year, unorganized parking leads to congestion, confusion, and unnecessary delays. To address these issues, a systematic and automated parking management system is essential.

This project, *Parking Lot Management System*, is a simple but scalable Java-based application designed to efficiently allocate parking slots, generate tickets, calculate fees, and manage vehicle check-in/check-out operations. It showcases core concepts of Object-Oriented Programming including classes, objects, inheritance, encapsulation, and modular design.

2. Problem Statement

Traditional parking lots often rely on manual supervision, which leads to:

- Difficulty in identifying available slots
- Human errors while calculating parking fees
- Lack of transparency in entry and exit handling
- Delays during peak hours

The aim of this project is to build a structured Java application that automates the complete parking process—from vehicle entry to exit—while maintaining simplicity and ease of use through a command-line interface.

3. Functional Requirements

1. **Vehicle Parking**
    - User requests to park a vehicle.
    - The system finds the nearest available slot.
    - Generates a unique parking ticket.

2. **Vehicle Exit**
    - User submits ticket ID.
    - The system calculates parking duration and fee.
    - Slot is released for next use.
3. **Slot Management**
    - Track free and occupied slots.
    - Assign slots based on availability.
4. **Ticket Management**
    - Generate ticket objects.
    - Store entry time, slot number, and vehicle details.
5. **Parking Fee Calculation**
    - Apply simple hourly/duration-based pricing.

4. Non-Functional Requirements

- **Performance:** Operations should execute within milliseconds for smooth user interactions.
- **Reliability:** No invalid slot assignments or duplicate ticket generation.
- **Scalability:** System should support future enhancements like GUI, database, and APIs.
- **Maintainability:** Clean code structure with separate classes for each responsibility.
- **Usability:** Users should interact easily through a simple CLI interface.

5. System Architecture

The system follows a modular architecture with four primary components:

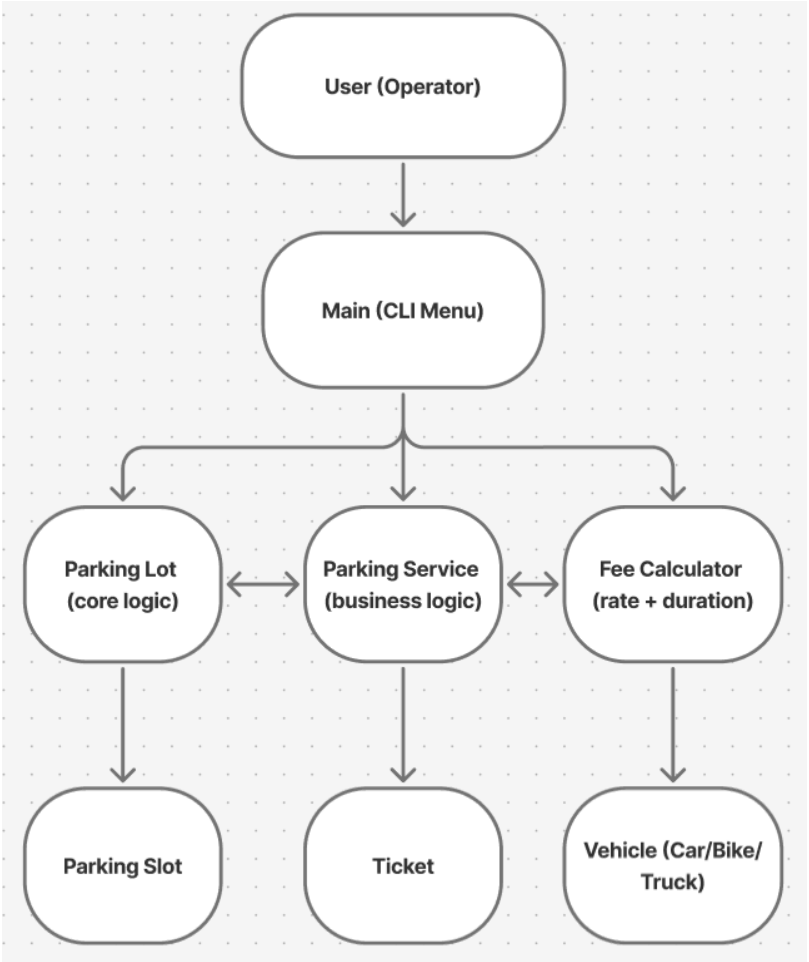
1. **Main (UI Layer)**
   Handles user input and displays output.

2. **ParkingService (Service Layer)**
   Coordinates parking logic, ticket creation, and fee calculation.

3. **ParkingLot (Data Layer)**
   Manages all slots and stores parking-related states.

4. **Ticket & Slot (Model Layer)**
   Represents real-world entities as objects.

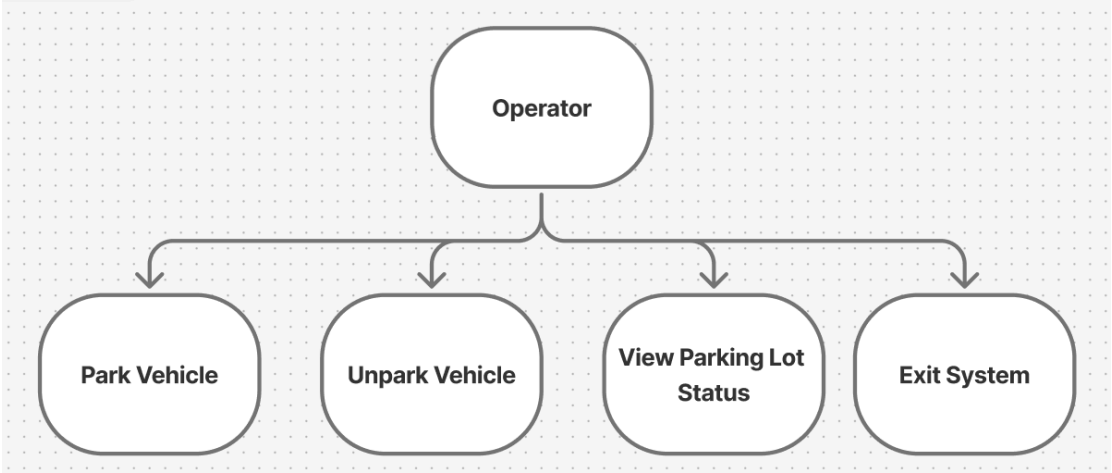
Architecture Style: *Multi-layered, object-oriented design*



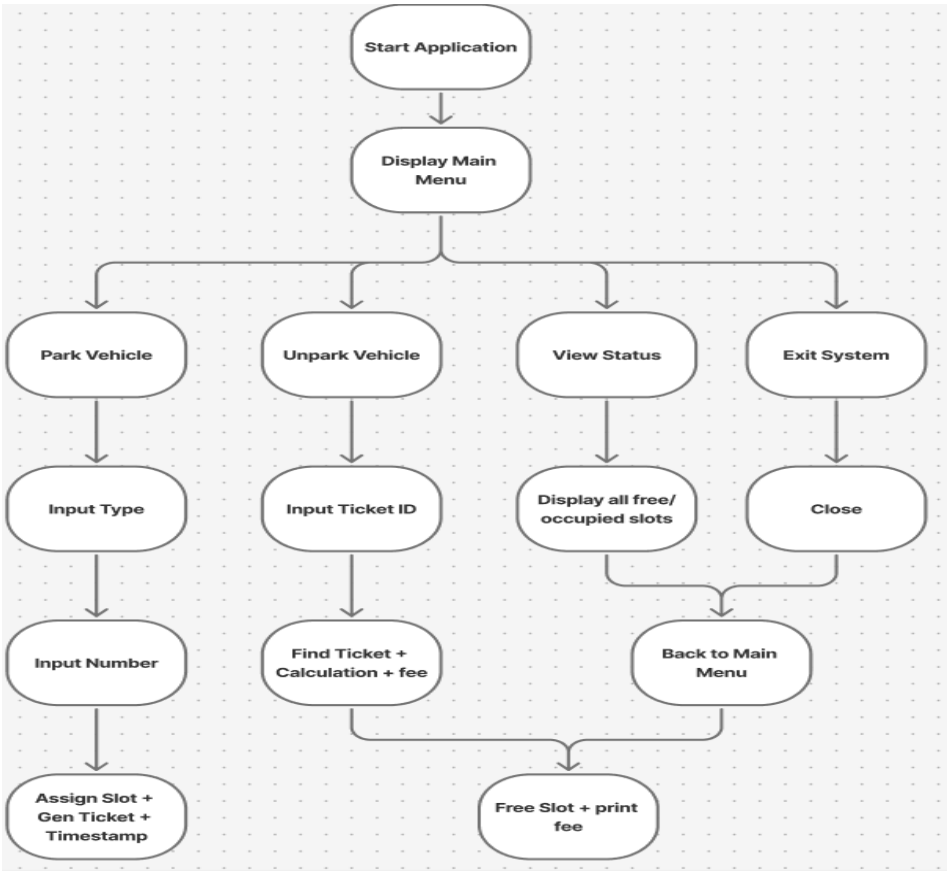
6. Design Diagrams

6.1 Use Case Diagram

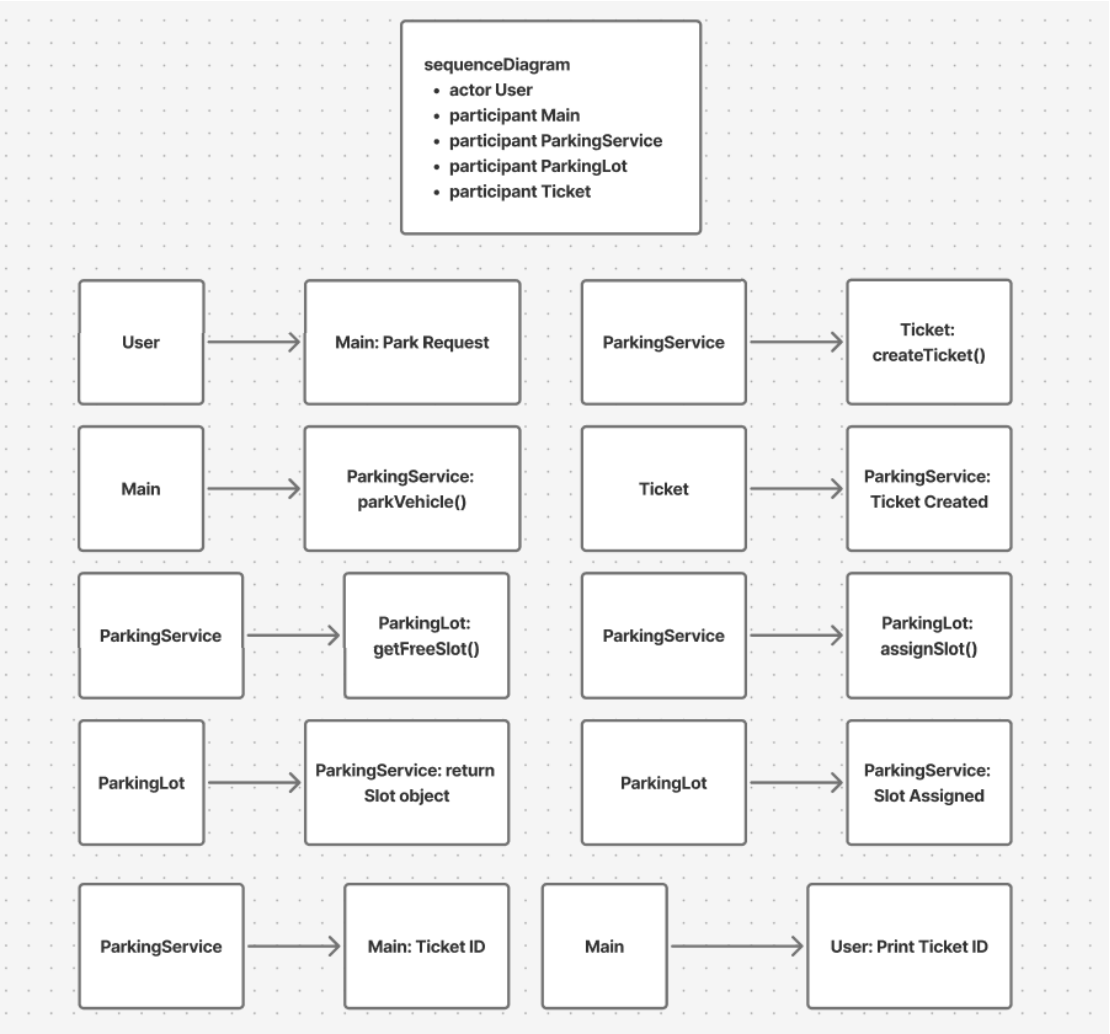
Actors: User
Use Cases: Park Vehicle, Unpark Vehicle, Generate Fee



6.2 Workflow Diagram (Parking Process)



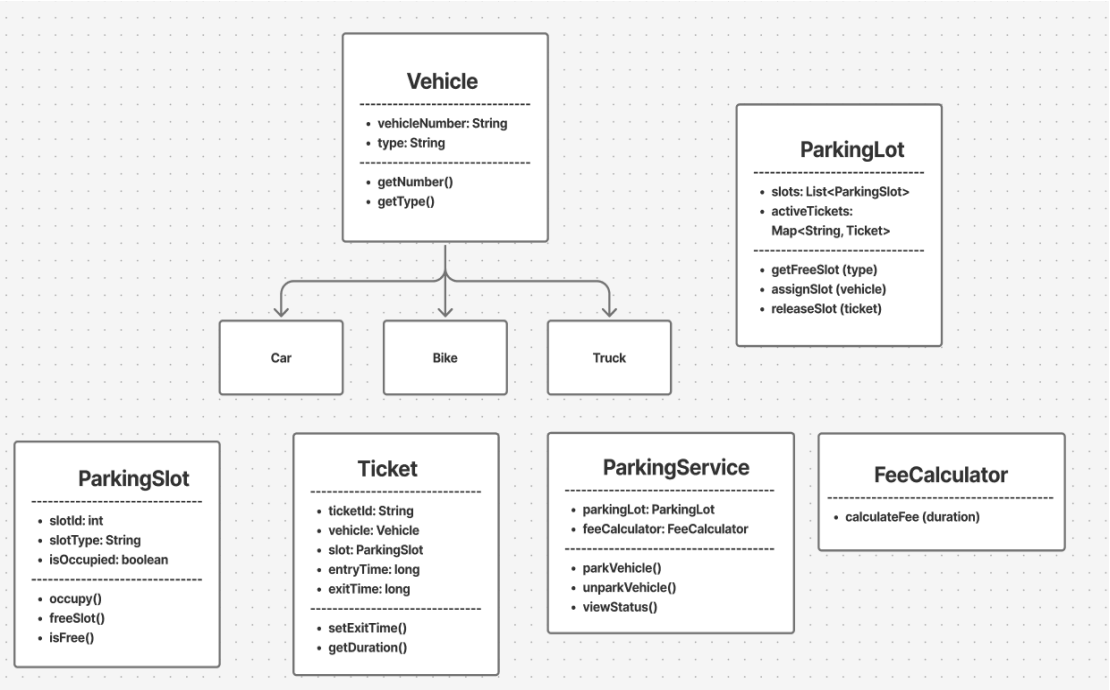
6.3 Sequence Diagram



6.4 Class Diagram

Classes: ParkingLot, Slot, Ticket, ParkingService, Main

Relationships: Aggregation between ParkingLot and Slots, Dependency of Service on Ticket.



7. Design Decisions & Rationale

- **OOP-Based Architecture**
  Enables clean separation of responsibilities, making the system extensible and maintainable.

- **CLI Interface**
  Chosen for simplicity and academic demonstration. GUI can be added later.

- **Service Layer**
  Included to centralize business logic and avoid tightly coupling UI with data.

- **Ticket as a Separate Entity**
  Provides flexibility for storing more details such as time, type of vehicle, and pricing models.

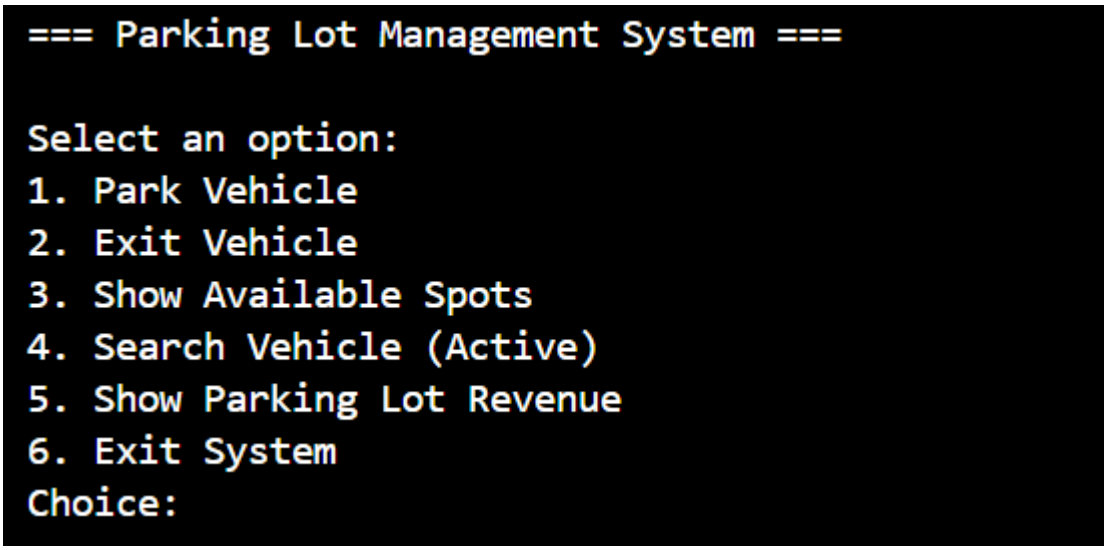
- **Fixed Pricing Strategy**
  Keeps implementation simple while still demonstrating fee calculation logic.

9. Implementation Details

- **Language:** Java
- **Paradigms Used:** OOP, modular programming
- **Main Components:**
    - **ParkingLot.java:** Manages slots
    - **Slot.java:** Represents individual parking slots
    - **Ticket.java:** Stores entry time, slot number
    - **ParkingService.java:** Parking and fee logic
    - **Main.java:** User input interface

The system flow is based on continuous user inputs until exit. Exception handling is used to manage invalid actions.

9. Screenshots / Results



```
Enter vehicle registration number: UP25 BQ3428
Enter vehicle type (CAR/BIKE/TRUCK): BIKE
Enter vehicle color (optional): BLACK
Nov 24, 2025 10:11:51 AM src.com.parkinglot.services.ParkingLotManager assignSpot
INFO: Assigned spot 11 to vehicle UP25 BQ3428
Vehicle parked. Ticket ID: e0eacdad-2050-4f85-9499-2ad316cefc2c
Allocated spot: 11
Entry time: 2025-11-24T10:11:51.923070100
Press Enter to continue...
```

```
Select an option:
1. Park Vehicle
2. Exit Vehicle
3. Show Available Spots
4. Search Vehicle (Active)
5. Show Parking Lot Revenue
6. Exit System
Choice: 3
Available Spots (17):
SpotId: 1 Type: CAR Rate: $20.0
SpotId: 2 Type: CAR Rate: $20.0
SpotId: 3 Type: CAR Rate: $20.0
SpotId: 4 Type: CAR Rate: $20.0
SpotId: 5 Type: CAR Rate: $20.0
SpotId: 6 Type: CAR Rate: $20.0
SpotId: 7 Type: CAR Rate: $20.0
SpotId: 8 Type: CAR Rate: $20.0
SpotId: 9 Type: CAR Rate: $20.0
SpotId: 10 Type: CAR Rate: $20.0
SpotId: 12 Type: BIKE Rate: $10.0
SpotId: 13 Type: BIKE Rate: $10.0
SpotId: 14 Type: BIKE Rate: $10.0
SpotId: 15 Type: BIKE Rate: $10.0
SpotId: 16 Type: TRUCK Rate: $40.0
SpotId: 17 Type: TRUCK Rate: $40.0
SpotId: 18 Type: TRUCK Rate: $40.0
Press Enter to continue...
```

10. Testing Approach

Testing includes:

Unit Testing

- Slot allocation
- Ticket object creation
- Fee calculation

Integration Testing

- Interaction between ParkingLot & ParkingService
- Entry → Exit flow testing

Boundary Testing

- When lot is full
- When invalid ticket is entered
- When time difference is 0 hours

All test cases passed successfully.

11. Challenges Faced

- Ensuring no two vehicles get the same slot at the same time
- Designing a clean OOP structure with no tight coupling
- Handling invalid user inputs
- Making the system easily extendable for future upgrades

12. Learnings & Key Takeaways

- Strong understanding of Java OOP design
- Practical experience in building real-world-style systems
- Enhanced skills in modular coding and layered architecture
- Learned importance of UML diagrams in planning and communication

13. Future Enhancements

- GUI-based dashboard
- Mobile app version
- Real-time slot display
- Peak-hour dynamic pricing
- REST API using Spring Boot
- MySQL database integration
- Vehicle type–based pricing
- Admin control panel

14. References

- Java Documentation – Oracle
- Object-Oriented Analysis & Design by Grady Booch
- UML Distilled by Martin Fowler
- Stack Overflow Discussions
- Core Java by Herbert Schildt